현재 구현과 제안 범위

FSM 기반은 공유하고, 이동 방식과 행동 판단은 분리한다. 현재 소스 확인 결과와 설계 제안을 구분하며, 이 문서는 코드 변경이나 구현 완료를 의미하지 않는다.

현재 Agent 구성

Agent	현재 State·실행 구성	행동 결정 경로
Player	Grounded · Jump · Fall · Dash · PlayerAttack · Hit · Death	PlayerInput → 입력 Transition → FSM
지상 Monster	Grounded · MonsterAttack · MonsterHit · Death	Behavior → MonsterInput → FSM
Flying Agent	Controller · Motor · MovementHandler · Data 기반	구체 Controller·Flight State·공격 연결 미구현

현재 공통 기반에 남아 있는 가정

현재 요소	소스에서 확인한 역할	분리 제안
AgentController 입력 참조	이동·공격 입력 Component를 공통으로 확보	입력을 사용하는 구체 Agent로 이동
AgentController 공격 처리	AgentCombatHandler 필수 부착·초기화, OnFrame에서 근접 판정	공격 구성·Event 처리 주체를 구체 Agent로 이동
GroundedAgentController	지면 갱신·지상 Motor·이동 Handler·점프 입력 확보	지면·이동 유지, 입력 의존성은 사용 주체로 이동

유지할 구조

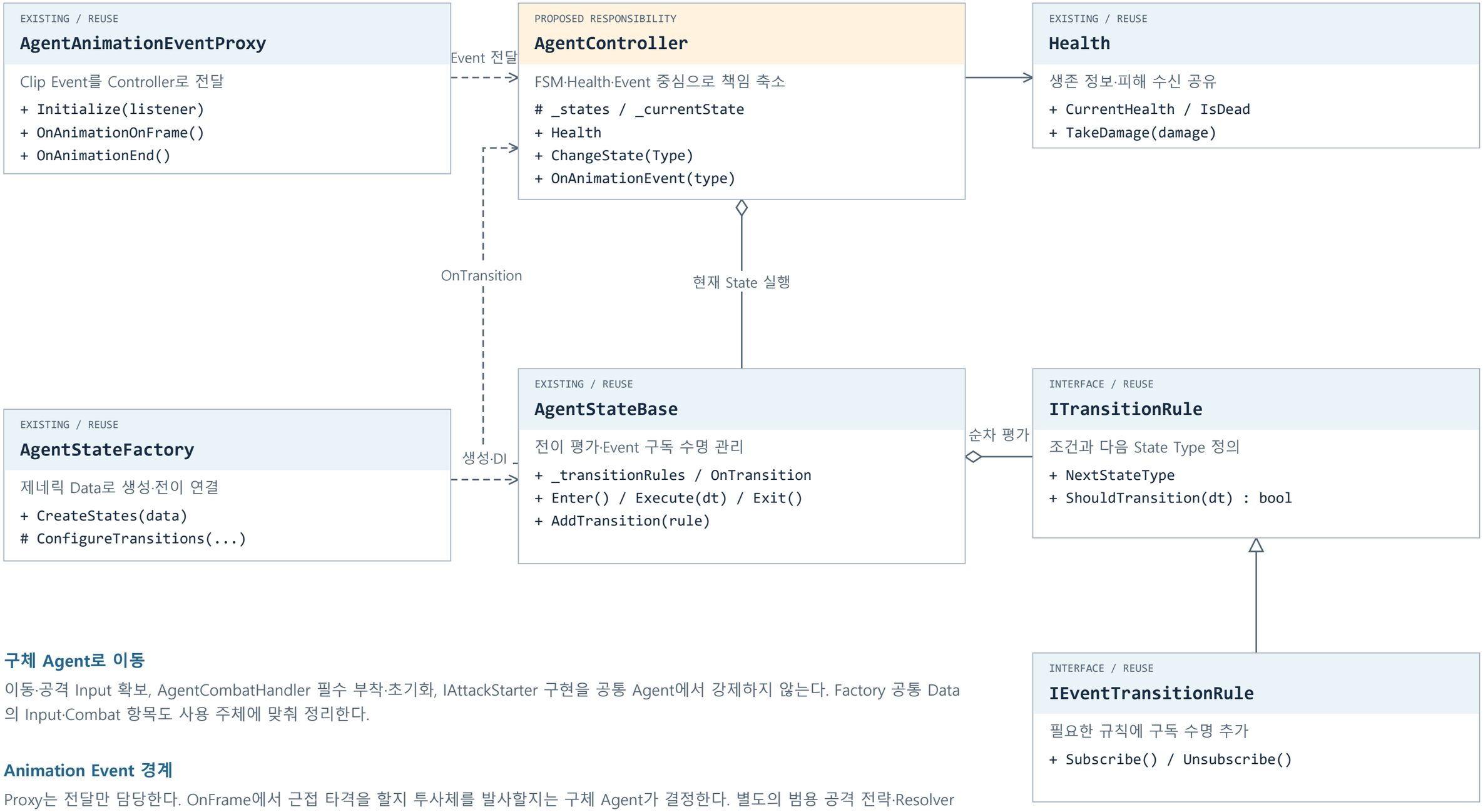
지상·비행 Motor와 이동 Data 분리, Animator의 공통 Hit·Death 및 개체별 capability 분리, State의 전이 평가·구독 수명 관리.

이번 제안의 범위

공통 기반을 더 크게 만들지 않는다. Player의 입력 규칙, Monster의 상황 판단, 향후 Boss의 패턴 진행은 개체별로 소유한다. Boss는 아직 구현하지 않았다.

공통 기반: FSM 실행과 생명주기

AgentController는 어떤 행동을 선택하는지가 아니라 State 실행·전환·Event 전달을 담당한다. 기존 FSM 수명 구조를 유지하면서 입력·특정 공격에 대한 필수 의존성을 줄이는 제안이다.



구체 Agent로 이동

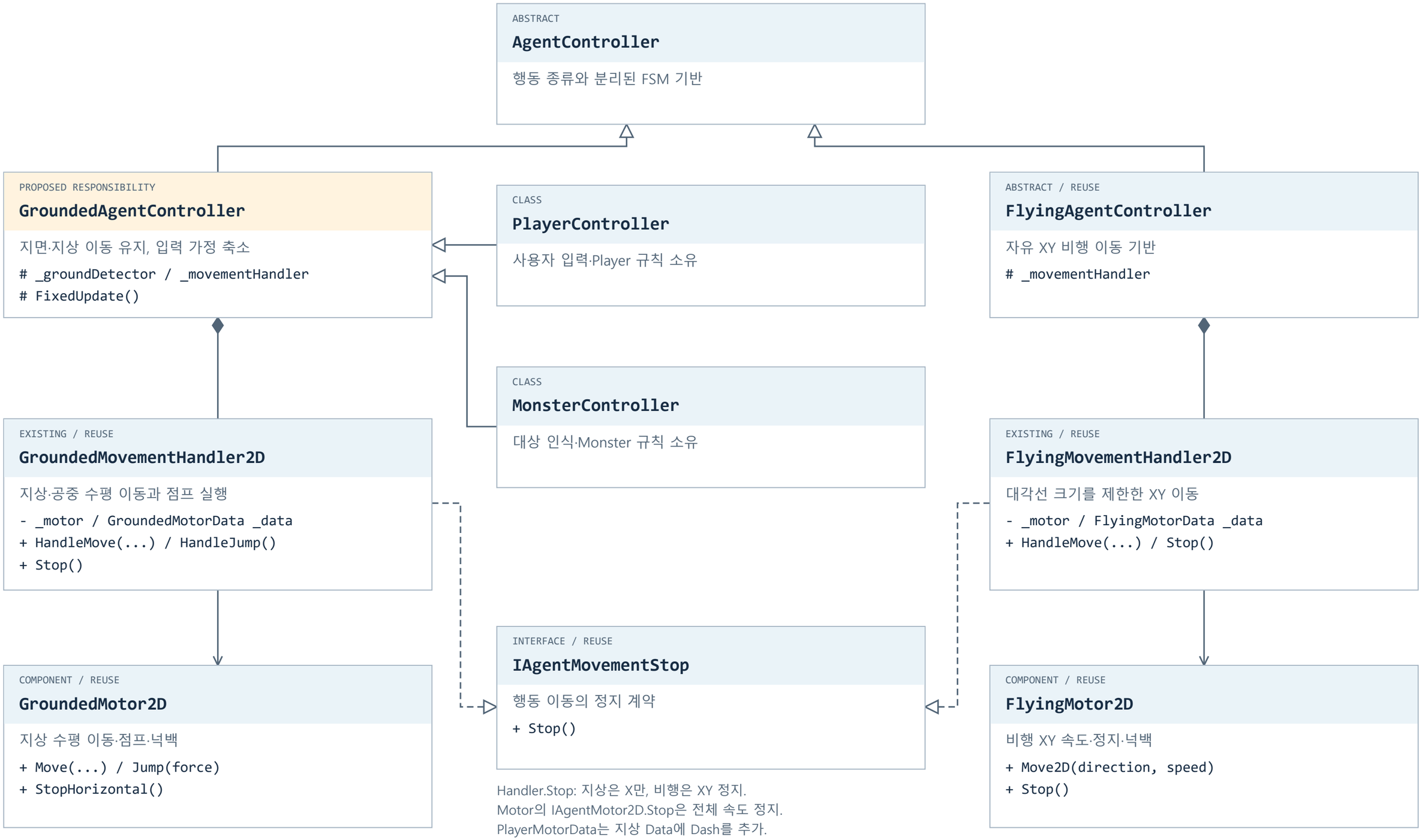
이동·공격 Input 확보, AgentCombatHandler 필수 부착·초기화, IAttackStarter 구현을 공통 Agent에서 강제하지 않는다. Factory 공통 Data의 Input·Combat 항목도 사용 주체에 맞춰 정리한다.

Animation Event 경계

Proxy는 전달만 담당한다. OnFrame에서 근접 타격을 할지 투사체를 발사할지는 구체 Agent가 결정한다. 별도의 범용 공격 전략·Resolver를 먼저 추가하지 않는다.

이동 방식: Grounded와 Flying

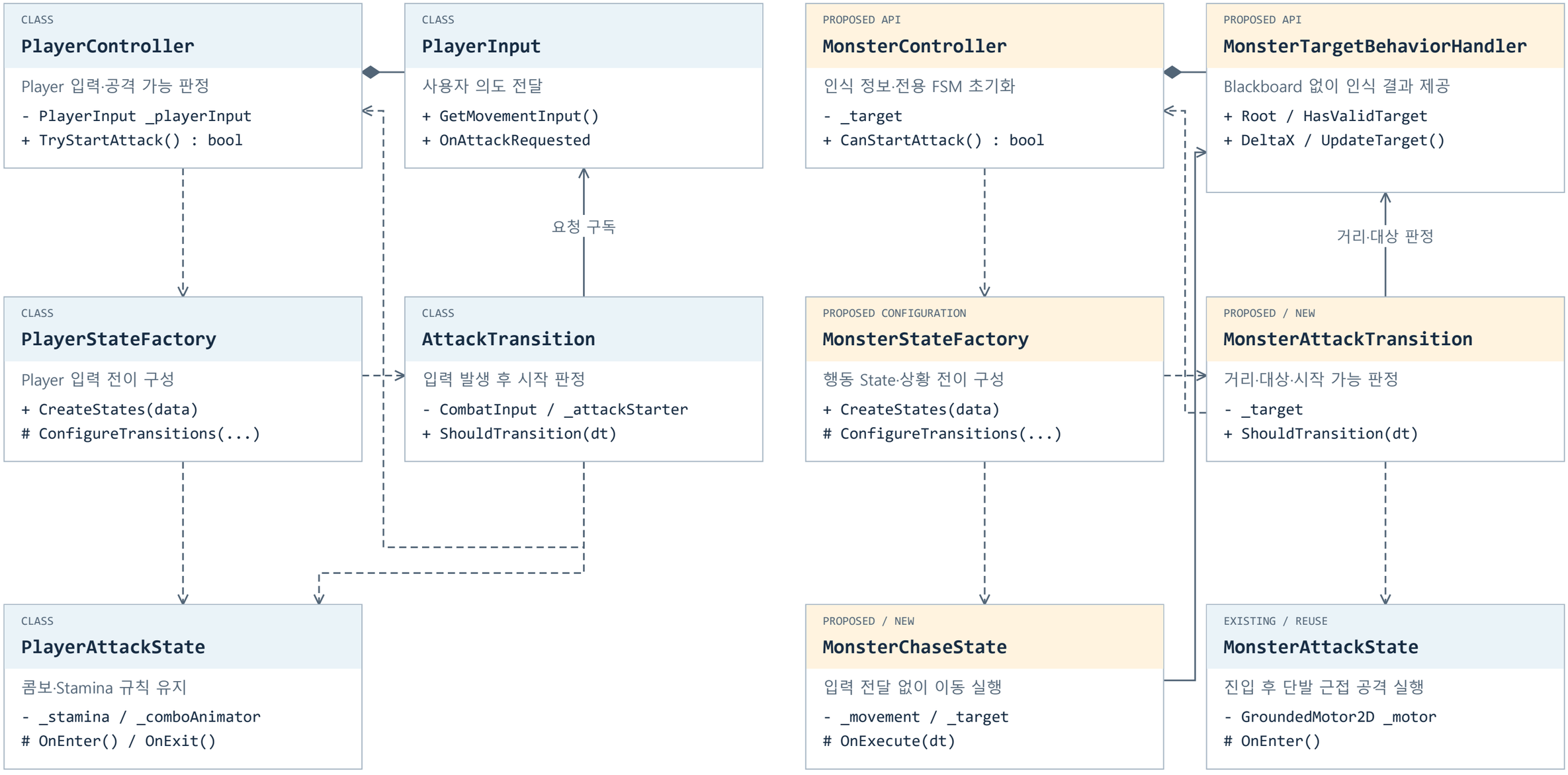
Player·AI 구분이 아니라 물리 이동 방식으로 계층을 나눈다. 현재 분리된 Motor·MovementHandler·Data를 유지하고 지상 공통 기반의 입력 가정을 줄인다.



주요 멤버만 표시 | 삼각형: 상속·구현 / 다이아몬드: 보유 / 점선 화살표: 생성·호출

판단 정책: Player와 Monster 분리

Player는 입력 전이를 유지한다. 단순 Monster는 대상 정보로 판단하는 전용 State·Transition을 사용한다. 노란색은 신규 State·전이 또는 기존 클래스의 API 변경 제안이다.



Player 경로 유지

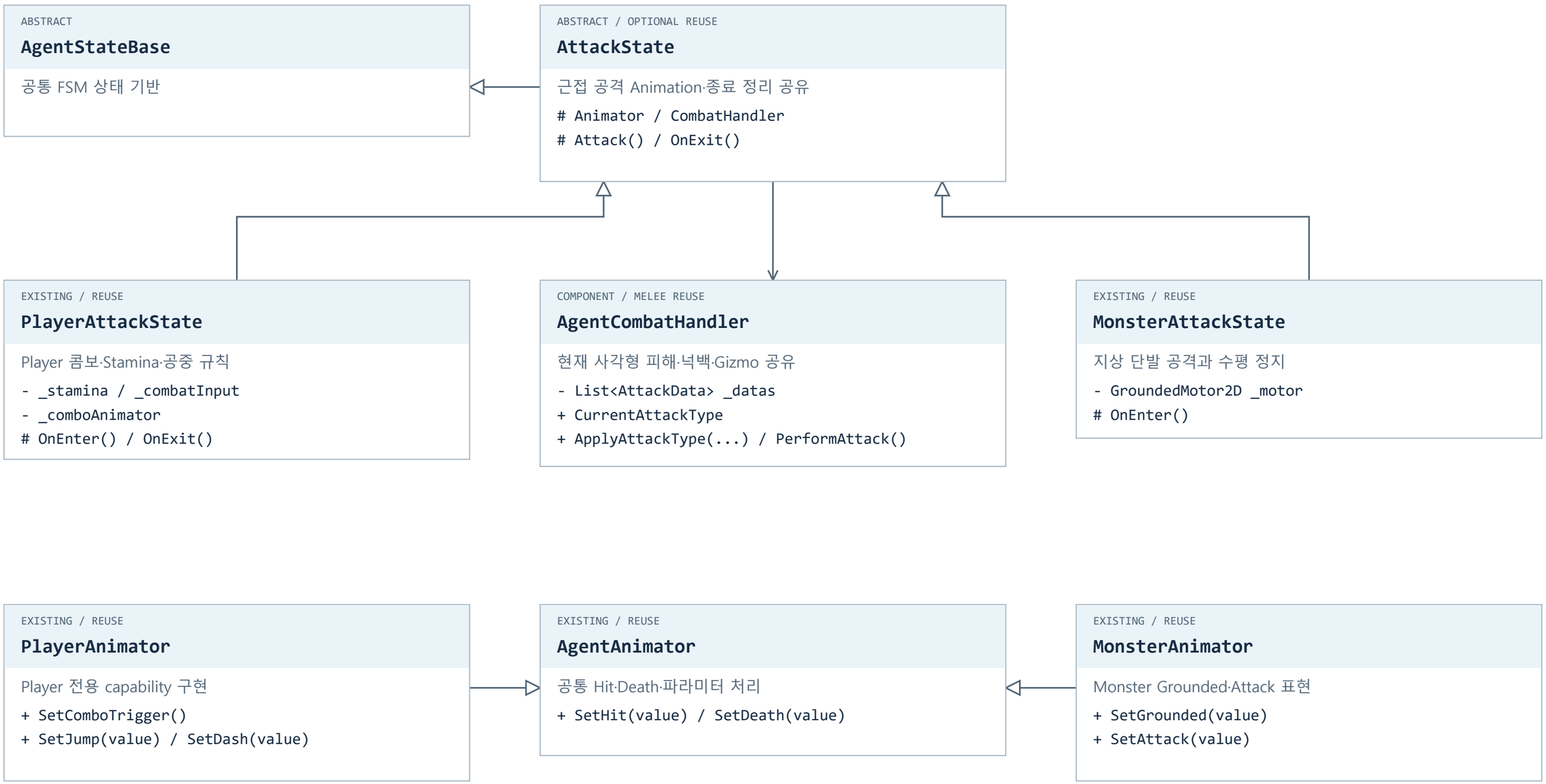
Grounded·Jump·Fall·Dash·Attack·Hit·Death. 사용자 입력은 Player 쪽에서 처리하고, 실제 공격은 State 진입에서 실행한다.

Monster 행동 State 제안

Patrol·Chase·Attack·LostTargetWait·Return·Hit·Death. Handler는 정보·기능, Transition은 선택, State는 실행을 담당한다. Behavior·MonsterInput 경로는 이 구성에서 제외한다.

공격·Animator: 동일한 부분만 공유

복구된 PlayerAttackState·MonsterAttackState 분리를 우선 유지한다. 아래 공격 기반은 현재 사각형 근접 공격 계열의 재사용 범위이며, 향후 모든 공격에 강제할 구조가 아니다.



공용화 범위

근접 공격의 동일한 피해 계산·Animation 수명만 공유한다. 콤보·Stamina는 PlayerAttackState에 유지하며, Wrapper 계층을 다시 도입하지 않는다.

특수 공격 확장

돌진·투사체가 실제로 필요해질 때 실행 구조를 별도로 설계한다. Animator도 사용하지 않는 기능을 AgentAnimator에 올리지 않고 필요한 전용 구현으로 유지한다.

공통화·분리의 판단 기준

공통 클래스 수를 늘리는 것이 아니라 공통 기반의 개체별 가정을 줄인다. 다음 경계를 기준으로 현재 코드의 재사용 여부를 판단한다.

공통으로 유지할 것과 개체별로 둘 것

영역	공통 유지	Agent별 책임
FSM	State 실행·전이 평가·구독 수명	State 구성·전이 우선순위·중단 조건
이동	지상·비행별 Motor와 Handler	입력·목적지·이동 의도 결정
공격	실제로 동일한 근접 피해 계산	공격 종류 선택·콤보·Stamina·특수 공격
Animation	공통 파라미터 처리·Hit·Death	Jump·Dash·Combo·종별 모션·Frame 처리
생존	Health·피해 수신 계약	게임 실패·오브젝트 제거 등 사망 후처리

단순 Monster

전용 FSM의 상황 Transition에서 행동을 결정한다. 공용 GroundedState 안에 순찰·추적 판단을 몰아넣거나, FSM 위에 별도 판단 Brain을 중복 추가하지 않는다.

Boss: 후속 선택

고정된 페이즈·패턴 순서는 패턴 진행기나 계층형 FSM, 지속적인 우선순위 재평가는 BehaviorTree를 검토한다. 지상·비행 기반은 보스 여부와 독립적으로 선택한다.

Boss 실행 결과 연결

Boss 판단은 공격 요청만 발행하고 끝내지 않는다. 시작 가능·실행 중·완료·중단 결과를 확인하여 다음 패턴을 선택한다. 모든 State 타입을 직접 비교하는 방식으로 강제하지 않으며, 필요한 API는 실제 패턴 설계 후 정한다.

소스 기준 및 문서 상태

Assets/Scripts/FSM/Agent/: @Hub/AgentController·AgentAnimator, StateControl/AgentStateBase·AgentStateFactory, Handler/AgentCombatHandler
GroundedAgent/: @Hub·Handler·SOData · FlyingAgent/: @Hub·Handler·SOData
Player/: @Hub·Input·PlayerState · NPC/AIMonstor/: @Hub·Input·MonsterState·@Behavior
2026-10-14 현재 소스 기준. 노란색 클래스·API와 Monster 행동 FSM 전환은 제안이며, 코드·Prefab·Graph에는 적용하지 않았다.